Solace Engine: Техническая документация

Часть I: Спецификация боевой системы и ИИ | Часть II: План рефакторинга рендеринга (Переход к C+)

ЧАСТЬ I. ЕДИНАЯ СПЕЦИФИКАЦИЯ БОЕВОЙ СИСТЕМЫ И ИИ

1. Общее видение и фундаментальные принципы

Проект представляет собой 3D-экшен в жанре темного фэнтези с элементами выживания и прогрессии силы. Боевая система полностью исключает неконтролируемый «спам кнопок» (button mashing) и строится вокруг пространственной физики, таймингов, оценки ордера противников и микро-решений.

- **Честная векторная физика и инерция:** Все персонажи и противники обладают физической массой, зависящей от их экипировки. Рывки и уклонения (дэши) представляют собой векторные импульсы движения, меняющие позицию и сбивающие прицел врагов, без кадров абсолютной неуязвимости (i-frames).
- **Прецизионные хитбоксы:** Просчет контактов выполняется на основе траектории движения лезвия/оружия (Swept-Hitboxes) в файле `combat_system.cpp`. Ошибка в дистанции ведет к промаху, фазе восстановления или застреванию оружия в окружении.
- **Бескомпромиссная цена ошибки:** Каждое физическое действие расходует выносливость. Истощение стамины переводит субъекта в состояние оглушения (stagger), открывая возможность для нанесения критического удара.
- **Адаптивный командный интеллект:** Противники объединены в общую сеть управления отрядом (`Squad Director`) и адаптивно подстраиваются под тактику игрока.

2. Ступенчатая прогрессия (Путь от Новичка к Мастеру)

Этап	Профиль противников	Динамика и ключевые механики	Состояние игрока
1. Шоковая адаптация	Дикие звери, примитивные существа.	Высокая летальность. Медленные анимации, дефицит сил. Выманивание целей по одной.	Уязвимый выживший
2. Порог цивилизации	Мелкие мародеры, бандитские группы.	Примитивное оружие. Враги зажимают с двух сторон. Использование узких проходов.	Начинающий боец
3. Тактическое пробуждение	Организованные наемники, щитоносцы.	Появление векторных дэшей. Чтение микро-телеграфов позы врага. Наказание в фазах recovery.	Закаленный тактик
4. Адаптивный перелом	Смешанные отряды (лучники, маги).	ИИ активирует динамическую память (`Squad Blackboard`) и профилирование игрока.	Опытный командир
5. Профессиональный строй	Армейские когорты, фаланги.	Высокие скорости боя. Использование комбо-цепочек, ломка стойки тяжелыми импактами.	Мастер клинка

Этап	Профиль противников	Динамика и ключевые механики	Состояние игрока
6. Катаклизм	Легионы элиты, массовые волны.	Полномасштабные столкновения. Режим Overclock (замедление времени), ультимативные навыки.	Живое бедствие
7. Сингулярность	Боссы-зеркала с симметричными навыками.	Высокоскоростной поединок с противниками, использующими микро-парирование и перехват.	Дуэль равных

3. Архитектура Командного ИИ (Squad Director)

Противники действуют под управлением глобального оркестратора группы (`Squad Director`), обновляющего данные в общем контейнере состояния (`Squad Blackboard`):

- **Сенсорная модель:** Конус зрения (с учетом освещенности и укрытий) и акустическая разведка (реакция на шаги, обнажение стали, шум).
- **Роли отряда:** `Front Anchor` (провокация блока), `Flanker` (обход в слепые зоны), `Suppressor` (дистанционный контроль), `Disrupter` (сбивание замаха башем), `Commander` (баффы и перегруппировка).
- **Управление агро (Token Bucket System):** Ограничение числа одновременно наносящих урон юнитов для предотвращения слияния хитбоксов.
- **Player Profiling:** Анализ повторяющихся действий игрока и выработка контрмер ИИ за задержку в 3–5 секунд.

4. Висцеральный импакт и 3D AA-Пайплайн

- **Hit-Stop:** Фиксация кадров на 2–8 фреймов в момент контакта клинка с целью для передачи плотности материала.
- **Bone Squashing:** Процедурное локальное сжатие костей скелета в точке попадания тупого оружия.
- **Система Кровавого Ритма:** Заполнение шкалы Flow State за счет идеальных парирований с выходом в режим *Overclock* (замедление времени).
- **Стилизованный 3D-рендеринг:** Использование мид-поли моделей с кастомным динамическим освещением и пост-обработкой для достижения 60+ FPS.

ЧАСТЬ II. ПЛАН РЕФАКТОРИНГА РЕНДЕРИНГА И ПЕРЕХОДА К C+

1. Введение, цели и обоснование C+

Целевое состояние графической подсистемы — архитектура уровня C+, сочетающая полную ECS-парадигму с оптимизацией конвейера рендеринга. В отличие от чистого ECS (C), уровень C+ вводит двухпроходную очередь отрисовки, формализацию материалов и централизованный пул буферов костей.

> **Ключевые принципы рефакторинга:**
> • *Изоляция API:* Вызовы OpenGL допускаются исключительно внутри `engine/graphics/renderer.cpp`.
> • *Разделение ответственности:* Scene отвечает за инициализацию; Systems — за логику; Renderer — за GL-команды.
> • *Единый пул костей:* Использование `BoneBufferPool` (SSBO) вместо отдельных буферов на каждый экземпляр.

2. Ключевые структуры данных и ECS-компоненты

- **DrawCall:** Содержит указатели на `MeshResource`, `Material`, матрицу `model`, идентификатор/смещение буфера костей, `AABB` и ключи сортировки.
- **RenderQueue:** Хранит векторы `m_opaque` (сортировка front-to-back по материалам) и `m_transparent` (back-to-front по дистанции).
- **RenderComponent & AnimationComponent:** ECS-компоненты, содержащие ссылки на ресурсы, индексы в пуле костей и параметры видимости.

3. Сравнительный анализ архитектуры

Аспект	Текущее состояние (Legacy)	Целевое состояние (C+)
Логика Scene::on_render	~80 строк с прямыми вызовами GL	3–5 строк (передача view-структуры)
Вызовы gl* в game/	Присутствуют повсеместно	Полностью отсутствуют
Представление сущностей	ООП-классы (Enemy, Player)	ECS Entity + набор компонентов
Управление костями	Per-instance SSBO на экземпляр	BoneBufferPool + glBindBufferRange
Оптимизация вызовов	Отсутствует (до 500 draw calls)	RenderQueue batching + Instancing
Отсечение геометрии	Отсутствует	Frustum Culling на базе AABB

4. Детализированный график миграции

Фаза	Наименование этапа	Сроки	Уровень риска
Фаза 0	Подготовка фундамента, очистка технического долга	1–2 часа	Низкий
Фаза A	Выделение изолирующего шва (RenderSystem)	2–3 часа	Низкий
Фаза B	Инкапсуляция Renderer, внедрение RenderQueue и Material	1.5–2 дня	Средний
Фаза C	Переход на полный ECS и централизованный BoneBufferPool	4–6 вечеров	Высокий
Фаза C+	Frustum Culling, Material Batching и Instancing окружения	3–4 вечера	Низкий–Средний

ЧАСТЬ III. СТРУКТУРА ПРОЕКТА И МАТЕМАТИЧЕСКАЯ МОДЕЛЬ

1. Структура модулей и файлов проекта

Каталог / Модуль	Файл	Зона ответственности
`engine/include/engine/ai/`	`behavior_tree.hpp`	Деревья решений, тактические узлы, сенсоры слуха/зрения.
`engine/include/engine/ai/`	`enemy_modules.hpp`	Модули ролей ИИ (Anchor, Flanker, Suppressor, Disrupter).
`engine/include/engine/systems/`	`combat_system.cpp`	Расчет Swept-Hitboxes, инерции массы, Hit-stop, Bone Squashing.
`engine/include/engine/systems/`	`item_system.hpp`	Система сборок, чипов памяти, статов и усилений.
`engine/include/engine/graphics/`	`renderer.cpp`	Единственная точка вызова OpenGL API, менеджмент SSBO.
`game/include/game/entities/`	`player.hpp`	Расчет физической инерции, стамины и вектора дэшей.
`game/include/game/world/`	`world_gen.cpp`	Процедурная генерация проходов, арок и разрушаемых объектов.

2. Математические модели и формулы (combat_system.cpp)

Расчет вектора скорости с учетом массы экипировки:

$$V_{new} = V_{current} + (F_{dash} / m_{armor}) \times \Delta t$$

Swept-Hitboxes & Рендер-пайплайн: Расчет пересечения векторов движения оружия между кадрами t_{-1} и t_{0} выполняются методом трассировки лучей. Отрисовка сцены осуществляется во внутренний FBO с фиксированным вертикальным разрешением (180p) с последующим масштабированием кастомным целочисленным шейдером (`post_fs.glsl`).

3. Регламент проверки регрессии (Checklist)

После выполнения каждой фазы миграции обязательна проверка по следующему регламенту:

1. Успешная сборка проекта через CMake в конфигурации Debug (`windows-vs-debug`).
2. Проверка базового игрового процесса: перемещение (WASD), атака, преследование ИИ врага, изменение HP, переключение анимаций (Idle/Run).
3. Проверка горячей перезагрузки (F5): корректность обновления параметров шейдера (`rimColor`) и разрешения FBO (`targetHeight`).
4. Контроль логов: отсутствие предупреждений *'Shader: uniform not found'* и ошибок графического API *'GL error'*.
5. Создание атомарного коммита в системе контроля версий.